

Pre-Norm vs Post-Norm.

Where you put the LayerNorm — *inside* the residual branch (pre-norm) or *wrapping* the residual sum (post-norm) — decides whether a deep transformer trains effortlessly or needs a delicate warmup schedule. Below we differentiate both placements, show why pre-norm keeps a clean identity gradient, and chart gradient-norm growth across 12 layers — every number computed.

THE EQUATION

$$\text{pre: } x' = x + S(\text{LN}(x))$$

$$\text{post: } x' = \text{LN}(x + S(x))$$

Worked example. An illustrative 12-layer backprop: pre-norm gradient norm grows gently to $\approx 1.80\times$; post-norm compounds to $\approx 14.55\times$, the explosion that motivates learning-rate warmup.

Topic 4 of 5 in this module · companion to the course page · v1.0

Contents

1	Two placements of one LayerNorm	2
2	Schematics	3
3	Differentiating both: where the identity gradient goes	3
4	Numeric comparison: gradient-norm growth over 12 layers	3
5	Consequences that matter	4
6	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 Two placements of one LayerNorm

A block has a sublayer S (attention or FFN), a residual add, and a LayerNorm. There are two canonical ways to wire them:

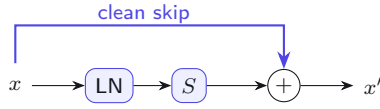
$$\boxed{\text{post-norm: } x' = \text{LN}(x + S(x)) \quad \text{pre-norm: } x' = x + S(\text{LN}(x))} \quad (1)$$

The original Transformer (Vaswani et al., 2017) used *post-norm*: normalize *after* adding the residual. Modern LLMs (GPT-2 onward, LLaMA, etc.) use *pre-norm*: normalize the input to the sublayer, leave the residual sum un-normalized.

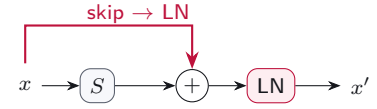
Symbol	Shape	Meaning
x, x'	$\mathbb{R}^d$	block input / output
$S(\cdot)$	$\mathbb{R}^d \rightarrow \mathbb{R}^d$	sublayer (attention or FFN)
LN	$\mathbb{R}^d \rightarrow \mathbb{R}^d$	layer normalization, Jacobian J_{LN}
J_S	$\mathbb{R}^{d \times d}$	sublayer Jacobian

In pre-norm, the skip wire carries the raw x straight from input to output with nothing in its way — a clean express lane. In post-norm, that same skip wire is forced through the LayerNorm at the end of *every* block, like a toll booth on each segment of the highway. One toll booth is fine; twelve in a row reshape the traffic, and the gradient that flows backward through them compounds.

2 Schematics



pre-norm: skip bypasses LN



post-norm: LN wraps the sum

3 Differentiating both: where the identity gradient goes

Pre-norm. Differentiate $x' = x + S(\text{LN}(x))$:

$$\frac{\partial x'}{\partial x} = I + J_S J_{\text{LN}}.$$

The leading I comes from the bare x and is *not* multiplied by any LayerNorm Jacobian. Back-propagating through L layers,

$$\frac{\partial \mathcal{L}}{\partial x_0} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=1}^L (I + J_{S_l} J_{\text{LN},l}),$$

and the all-identity term I survives undiminished — the gradient highway of Topic 1, intact.

Post-norm. Differentiate $x' = \text{LN}(x + S(x))$. Let $u = x + S(x)$; then

$$\frac{\partial x'}{\partial x} = J_{\text{LN}} \cdot \frac{\partial u}{\partial x} = J_{\text{LN}} (I + J_S).$$

Now the identity term is *premultiplied* by J_{LN} . Over L layers,

$$\frac{\partial \mathcal{L}}{\partial x_0} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=1}^L J_{\text{LN},l} (I + J_{S_l}),$$

so *every* backward step passes the gradient through a LayerNorm Jacobian. There is no clean I -only path: the would-be highway runs through L consecutive LN toll booths, whose Jacobian norms multiply.

pre: $I + J_S J_{\text{LN}}$ (clean I) post: $J_{\text{LN}}(I + J_S)$ (no clean I)	(2)
--	-----

The difference is *one matrix factor*. Pre-norm keeps an un-scaled identity gradient at every layer, so deep stacks train stably from step one — no warmup needed. Post-norm scales the identity by J_{LN} at every layer; these factors compound, the gradient at initialization is large and uneven across depth, and a learning-rate *warmup* (plus careful initialization) is needed to avoid early divergence.

4 Numeric comparison: gradient-norm growth over 12 layers

We model the relative gradient norm $\|\partial \mathcal{L} / \partial x_{L-k}\|$ as we backprop k layers down from the top, starting at 1.0. In the illustrative regime each pre-norm backward step multiplies by a gentle

$(1 + 0.05)$, while each post-norm step multiplies by $J_{\text{LN}}(I + J_S)$ whose norm is $\approx (1 + 0.25)$ at initialization. So

$$\text{pre}_k = 1.05^k, \quad \text{post}_k = 1.25^k.$$

layers from top k	pre-norm 1.05^k	post-norm 1.25^k
1	1.0500	1.2500
2	1.1025	1.5625
3	1.1576	1.9531
4	1.2155	2.4414
5	1.2763	3.0518
6	1.3401	3.8147
7	1.4071	4.7684
8	1.4775	5.9605
9	1.5513	7.4506
10	1.6289	9.3132
11	1.7103	11.6415
12	1.7959	14.5519

After 12 layers, post-norm has amplified the gradient $14.55\times$ versus pre-norm’s modest $1.80\times$ — an $8\times$ gap that widens with depth. Heatmap (intensity scales with magnitude, cap 80):

	$k=2$	$k=4$	$k=6$	$k=8$	$k=10$	$k=12$
pre	1.10	1.22	1.34	1.48	1.63	1.80
post	1.56	2.44	3.81	5.96	9.31	14.55

5 Consequences that matter

1. **Pre-norm trains without warmup.** The clean identity path keeps gradients well-conditioned, so a constant or simply-decayed learning rate works even for very deep stacks.
2. **Post-norm needs warmup + care.** Large early gradients force a ramp-up schedule and careful initialization to avoid divergence.
3. **Trade-off.** Post-norm, once tamed, can reach slightly stronger final performance because every output passes through a normalizer; pre-norm trades a little of that for robustness and is the default in large-scale LLMs.
4. **Final norm.** Pre-norm stacks add one extra LayerNorm at the very end (the running residual sum is otherwise never normalized).

“Pre-norm” does not mean “no normalization on the output.” Because the residual sum in a pre-norm stack is never normalized internally, its magnitude grows with depth; pre-norm architectures therefore append a *single final* LayerNorm before the output head. Forgetting it leaves the logits mis-scaled. Conversely, do not bolt a per-layer post-norm

onto a pre-norm design — you reintroduce the compounding J_{LN} factors you were trying to avoid.

6 Where this sits in the track

Link	Connection
Topic 1 (this module)	The residual gradient highway — pre-norm preserves it, post-norm scales it.
Topic 2 (this module)	J_{LN} , the LayerNorm Jacobian that does the scaling.
Phase 1	GPT-2/LLaMA use pre-norm; the original Transformer used post-norm — a direct consequence of (2).

A Reproducible (NumPy)

Inputs: pre-norm per-layer factor 1.05, post-norm per-layer factor 1.25, depth 12. Outputs: pre $\rightarrow$ 1.7959, post $\rightarrow$ 14.5519.

```
import numpy as np
k = np.arange(1, 13)
pre = 1.05 ** k      # gentle accumulation, clean identity path
post = 1.25 ** k     # compounded LN-Jacobian on the identity path
for i in k:
    print(i, round(pre[i-1], 4), round(post[i-1], 4))
print(pre[-1], post[-1]) # 1.7958563... 14.5519152...
print(post[-1] / pre[-1]) # ~8.10x gap after 12 layers
```